LoRA: Low-Rank Adaptation of Large Language Models

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Weizhu Chen

Abstract

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to specific tasks. As models grow, full fine-tuning becomes computationally prohibitive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. LoRA reduces the number of trainable parameters by 10,000 times and GPU memory requirement by 3 times compared to GPT-3 175B fine-tuning. LoRA performs on-par or better than full fine-tuning on RoBERTa, DeBERTa, and GPT-3, while having higher throughput and no additional inference latency.

1. Introduction

Many applications rely on adapting one large, pre-trained language model to multiple downstream tasks. Full fine-tuning requires updating all parameters of the model. When deploying multiple adapted instances, storing multiple copies of multi-billion parameter models is prohibitively expensive. LoRA addresses this by decomposing parameter updates into low-rank matrices $W = W_0 + B * A$, where $B \in R^{d \times r}$ and $A \in R^{r \times k}$ with rank $r << \min(d, k)$.

2. Methodology and Parameter Efficiency
In LoRA, during training W_0 is kept frozen and does not receive gradient updates. A Gaussian initialization is used for A and zero for B, so Delta W = B * A is zero at the beginning of training. We scale Delta W by alpha / r where alpha is a constant. When applied to GPT-3 175B, setting rank r = 4 results in only 37.7 million trainable parameters out of 175 billion (0.02% of parameters). Memory consumption during checkpointing drops from 350 GB to under 35 MB per task checkpoint.

3. Empirical Findings on GLUE and SuperGLUE
On the GLUE benchmark, LoRA matched or exceeded full fine-tuning accuracy across all tasks. Specifically, LoRA achieved 89.2% accuracy on MNLI and 92.4% on SST-2, matching full fine-tuning within 0.1% while requiring only 1/10,000th of the storage. 4. Limitations
A limitation of LoRA is that batching different tasks together in a single forward pass requires dynamically selecting weight branches, which introduces engineering complexity in high-throughput production servers.